Solution Architecture

Project Design Phase

Project Name	Automated Network Request Management in ServiceNow
Phase	Phase 3: Project Design
Author	Srikakulam Srikanth Surya
Team ID	SRIKANTHSURYA-NRM-01
Date	29 June 2026
Platform / Domain	ServiceNow / IT Service Management (ITSM)
Academic Year	2025 – 2026

1. Conceptual Solution Architecture

The Automated Network Request Management system bridges the gap between disorganized enterprise request channels and structured IT infrastructure provisioning. Built entirely within the ServiceNow enterprise PaaS cloud, the architecture is designed around four primary processing layers that transform a business user's request into an auditable, completed network change.

Presentation Layer

ServiceNow Service Portal (Requester Interface) and ServiceNow Native ITSM Workspace (Engineer & Approver Interface). End users interact through consumer-grade portal widgets; engineers and approvers operate within native workspace views.

Application & Security Layer

UI Policies and Client Scripts enforce dynamic form behaviors and validate all input parameters (IP addresses, CIDR notation, port numbers, VLAN identifiers) at the point of entry. Access Control Lists (ACLs) enforce role-based read/write/create/delete restrictions across all data tables.

Automation Engine

Flow Designer orchestrates the full request lifecycle: conditional multi-stage approval routing (Line Manager → Network Security), automated sc_task fulfillment task generation, SLA clock management with escalation triggers, and event-driven email notification dispatch at every state transition.

Storage Layer

ServiceNow cloud database stores all operational data across standard ITSM tables (sc_req_item, sysapproval_approver, sc_task) and the custom u_network_database table for historical operational logging. All writes are immutably timestamped for audit and compliance purposes.

2. Architecture Data Flow

Step	Stage	Description
1	Form Submission	Requester submits a catalog item form via the Service Portal with all validated technical parameters.
2	Payload Processing	Client Scripts validate IP/VLAN/port format. UI Policies enforce mandatory fields. Validated payload written to sc_req_item.
3	Approval Routing	Flow Designer evaluates request type and risk classification. Low-risk → Line Manager only. High-risk → Line Manager then Network Security Team.
4	Approval Decision	Approver receives email with full context. Approves/rejects via workspace or email link. Decision recorded in sysapproval_approver with timestamp.
5	Task Generation	Upon final approval, Flow Designer auto-generates a sc_task record and routes it to the Network Operations Group queue via Assignment Rules.
6	SLA Activation	SLA clock starts upon sc_task creation. Automated escalation emails fire at 50%, 75%, and 100% of breach threshold.
7	Fulfillment	Network Engineer executes the change, updates the sc_task work notes with completion details, and closes the task.
8	Request Closure	Task closure triggers Flow Designer to close the parent sc_req_item and dispatch a completion notification to the requester.

3. Key Architectural Decisions

All-Native ServiceNow Architecture

The entire solution is built exclusively using native ServiceNow modules — no external integrations, middleware, or custom code deployments. This ensures full platform support, simplified maintenance, and native upgrade compatibility.

Risk-Based Conditional Routing

Flow Designer applies conditional logic to route requests dynamically based on risk classification. Firewall changes and VLAN modifications are automatically escalated for security review; lower-risk requests (DNS changes, bandwidth upgrades) bypass this step.

Client-Side Input Validation

Regex validation is applied at the Client Script layer before submission, preventing invalid network parameters from entering the system at all — reducing processing errors and clarification cycles.

Update Set Deployment Strategy

All configuration changes are packaged in ServiceNow Update Sets, enabling repeatable, auditable, and reversible migrations between development, test, and production instances.